

The row indexes start at 0 at the top of the board and increase to 6 at the bottom of the board. We check the top row in the selected column to see whether it's occupied. If so, this column is completely full and the `continue` statement moves the execution back to the start of the loop to ask the player for another move.

If the column isn't full, we need to find the lowest unoccupied space for the tile to land on:

```
# Starting from the bottom, find the first empty space.
for rowIndex in range(BOARD_HEIGHT - 1, -1, -1):
    if board[(columnIndex, rowIndex)] == EMPTY_SPACE:
        return (columnIndex, rowIndex)
```

This `for` loop starts at the bottom row index, `BOARD_HEIGHT - 1` or 6, and moves up until it finds the first empty space. The function then returns the indexes of the lowest empty space.

Anytime the board is full, the game ends in a tie:

```
def isFull(board):
    """Returns True if the `board` has no empty spaces, otherwise
    returns False."""
    for rowIndex in range(BOARD_HEIGHT):
        for columnIndex in range(BOARD_WIDTH):
            if board[(columnIndex, rowIndex)] == EMPTY_SPACE:
                return False # Found an empty space, so return False.
    return True # All spaces are full.
```

The `isFull()` function uses a pair of nested `for` loops to iterate over every place on the board. If it finds a single empty space, the board isn't full, and the function returns `False`. If the execution makes it through both loops, the `isFull()` function found no empty space, so it returns `True`.

The `isWinner()` function checks whether a player has won the game:

```
def isWinner(playerTile, board):
    """Returns True if `playerTile` has four tiles in a row on `board`,
    otherwise returns False."""

    # Go through the entire board, checking for four-in-a-row:
    for columnIndex in range(BOARD_WIDTH - 3):
```